

Annexe K, Questions to the Product Owner

One grouped salvo · 5 questions · Samuel, QA Automation Senior · 15/08/2026

Each question is tied to a decision I cannot make alone, and each states what changes in my strategy depending on the answer. “Not documented, assume it” is an acceptable answer; I will log the assumption and continue.

Q1, Is Idempotency-Key mandatory on POST /transfers, and is deduplication server-side?

Annexe F shows the header on both sample requests; Annexe E’s contract never mentions it; Annexe G extract B shows two keyless POSTs from user u3, 121 ms apart, both settled by the bank adapter.

Why it changes my strategy: if the header is mandatory, TS-03c becomes a P1 regression test and a blocking CI gate. If deduplication is instead meant to be server-side on a payload hash, I test a different property entirely, and the fix for the double debit sits in a different team’s backlog. Today I cannot tell whether Annexe G is a client bug or a missing server control, so I cannot route the defect.

Q2, What is “Total Balance” supposed to represent, and which system of record should supply it?

(Reframed after live verification. I originally asked which derivation was intended. That question no longer applies: the figure is not derived from anything.)

The dashboard shows **\$48,291.00**, the transactions page shows **\$50,563.80**, and the ledger sums to **-\$389.66**. The two displayed figures differ from each other by \$2,272.80. Both are static, cycling the status filter changes the row count correctly (15/10/3/2) while the balance stays frozen, so they are hard-coded literals, not computations.

Why it changes my strategy: I cannot write a reconciliation assertion against an undefined quantity. Tell me the intended semantics, available balance, settled balance, account balance from a core banking system, and TS-08b becomes a precise check. Until then the strongest thing I can assert is that two screens disagree with each other, which is a symptom rather than the rule being broken.

Q3, Is amount cents-as-integer (per Annexe E) or dollars-as-decimal (per the deployed API)?

The contract declares type: integer, minimum: 1, “Montant en cents”. The deployed /transactions returns signed doubles in major units, -127.85, 89.99, -2500.

Why it changes my strategy: this decides whether my R03 boundary values are 999999 or 9999.99, and whether every reconciliation assertion runs in integer cents or must tolerate float epsilon. It also decides whether I raise the defect against the contract or against the implementation. I have assumed cents (assumption H-02) and written all arithmetic in integer cents, **a wrong assumption here silently invalidates six tests**, which is worse than a failing one.

Q4, Which environment should QA validate against, and when will it exist?

(This replaces my original Q4, which asked whether the unauthenticated API access was a demo concession. It was, I answered it with evidence rather than spending a question on it. The demo backend is a json-server fixture: GET /db returns the entire database, and POST {"hello": "world"} returns 201.)

The consequence is that **no server-side business rule can be tested here**. Not idempotency, not the R03 ceiling, not tenant scoping, not the contract. Fourteen tests are written and have nothing to run against.

Why it changes my strategy: with a real backend I execute those fourteen tests in ten minutes and can re-attribute six findings that are currently logged as environment artefacts. Without one, my automation is limited to the Angular client and I cannot support any release decision that depends on server behaviour. Annexe H's TC-11/TC-12 failures on *preprod* suggest that environment runs a real backend, **can QA have access to it, and is it representative?**

Q5, Which team owns the bank-settlement webhook, and can I get a signing secret plus a replayable test event?

Annexe I records **89 signature-verification failures** in 24 hours. Annexe G extract C shows one event verified after an initial 401, then correctly no-op'd twice as `already_settled`.

Why it changes my strategy: with a secret and a stub endpoint I can automate the settlement path, forged signature, replayed event, out-of-order delivery — which is journey J5, currently at **zero coverage** on a trust boundary where an external party asserts that money moved. Without them, J5 stays a documented residual risk I can only reason about, and Annexe D's "confirmation < 2 s" target is unverifiable. I would also need to know whether the 89 failures come from one misconfigured sender or many sources, the extract shows a single `source_ip`, which hints at the former, but one log line is not a distribution.